

CAN 라이브러리 - Bluetooth기반 교통약자용 버스 전동문 및 전동경사로 소규모 통합 제어 시스템

```
1  # CAN 라이브러리 사용자 계층 인터페이스 명세
2
3  이 문서는 현재 CAN 라이브러리에서 **사용자가 실제로 직접 사용할 계층**을 중심으로
4  구현 내부인 `CANCore` 이하 계층은 전체 구조를 이해할 수 있을 정도로만 간략히 설명
5
6  문서의 기준은 다음과 같다.
7
8  - 가장 간단한 사용 경로: `CANSocket`
9  - 여러 작업을 정적으로 관리하는 상위 경로: `CANContext` / `CANExecutor` / `CANService`
10 - 내부 구현 계층: `CANCore` / `CANPlatform` / MCU Driver / BSP
11
12 ---
13
14 ## 1. 권장 사용 계층
15
16 실사용 관점에서는 아래처럼 생각하면 된다.
17
18 ### 1-1. 가장 쉬운 사용법
19 - **`CANSocket`**
20 - 단일 포트 열기 / 송신 / 수신 / poll / timeout 기반 동작을 가장 간단하게 쓸 수 있다.
21 - 대부분의 애플리케이션은 우선 이 계층부터 사용하면 된다.
22
23 ### 1-2. 여러 작업을 정적으로 굴리고 싶을 때
24 - **`CANContext` + `CANExecutor` + `CANService`**
25 - send / receive / poll 작업을 여러 개 준비해 두고, 정적 슬롯 안에서 순차적으로 호출한다.
26 - 비동기 프레임워크처럼 보이지만, 실제 모델은 **동적할당 없는 polling 기반 operation**이다.
27
28 ### 1-3. 내부 안정 계층
29 - **`CANCore`**
30 - 실제 송수신, 상태 전이, timeout-aware polling의 기준이 되는 안정 계층이다.
31 - 사용자도 직접 쓸 수는 있지만, 일반적인 앱 레벨에서는 `CANSocket` 또는 `CANContext`를 사용한다.
32
33 ---
34
35 ## 2. 전체 계층 구조
36
37 ```mermaid
38 flowchart TB
39     APP[Application]
40
41     subgraph USER[User-facing layer]
42         SOCKET[CANSocket]
43         SERVICE[CANService]
44         CONTEXT[CANContext]
45         EXECUTOR[CANExecutor]
46         OP[CANOperation]
47     end
48
49     subgraph CORE[Core layer]
50         CORENODE[CANCore]
51     end
52
53     subgraph PLATFORM[Platform layer]
54         PLATFORMNODE[CANPlatform]
55         GLUE[Board/backend specific Binding Glue]
56     end
57
58     subgraph MCU[MCU Driver layer]
```

```

59     DRIVER[can_tc3xx / can_tc275]
60 end
61
62 subgraph BSP[BSP / Board layer]
63     BOARD[board_api / board port map]
64 end
65
66 APP --> SOCKET
67 APP --> SERVICE
68 SERVICE --> CONTEXT
69 CONTEXT --> EXECUTOR
70 CONTEXT --> OP
71 SOCKET --> CORENODE
72 CONTEXT --> CORENODE
73 CORENODE --> PLATFORMNODE
74 PLATFORMNODE --> GLUE
75 GLUE --> DRIVER
76 PLATFORMNODE --> BOARD
77 ...
78
79 ### 계층별 역할 요약
80
81 - **Application**
82   사용자 코드.
83
84 - **CANSocket**
85   가장 짧고 쉬운 공개 인터페이스. 단순 송수신용.
86
87 - **CANService / CANContext / CANExecutor / CANOperation**
88   여러 작업을 정적 자원으로 관리하는 상위 orchestration 계층.
89
90 - **CANCore**
91   라이브러리의 안정 코어. 실제 send / receive / poll / timeout 처리 기준점.
92
93 - **CANPlatform**
94   포트 이름(`can0`)을 실제 보드 포트로 찾고, 해당 보드/백엔드에 맞는 **bindi
95
96 - **MCU Driver layer**
97   TC375면 `can_tc3xx`, TC275면 `can_tc275` 같은 실제 드라이버 래퍼.
98
99 - **BSP / Board layer**
100   보드별 포트 정의, 핀맵, PHY/termination 정보 제공.
101
102 ---
103
104 ## 3. `CANSocket` 계층
105
106 ## 3-1. 언제 쓰면 되는가
107
108 `CANSocket`은 다음 같은 경우에 적합하다.
109
110 - 포트 하나 열어서 바로 송신/수신하고 싶다.
111 - `open -> send/receive -> close` 흐름이 필요하다.
112 - 복잡한 operation pool 없이 가장 짧은 API를 쓰고 싶다.
113 - polling-first 스타일로 동작해도 된다.
114
115 즉, **일반적인 사용자용 1순위 API**라고 보면 된다.
116
117 ---
118
119 ## 3-2. 핵심 특징
120
121 - `CANCore` 위의 얇은 socket-like facade
122 - 동적할당 없음
123 - 두 가지 사용 방식 지원

```

```

124 - **owning open**: `CANSocketOpen()`으로 내부 core까지 직접 관리
125 - **non-owning bind**: 외부에서 만든 `CANCore`를 `CANSocketBindCore()`
126
127 가장 단순한 경우에는 owning open만 알면 된다.
128
129 ---
130
131 ## 3-3. 기본 수명주기
132
133 ### 가장 일반적인 흐름
134
135 1. `CANSocketInit()`
136 2. `CANSocketInitOpenParamsClassic500k()` 또는 `CANSocketInitOpenParams
137 3. `params.port_name = "can0"` 설정
138 4. 필요 시 runtime timeout hook 설정
139 5. `CANSocketOpen()`
140 6. `CANSocketSend()` / `CANSocketReceive()` / `CANSocketPoll()` 사용
141 7. `CANSocketClose()`
142
143 ---
144
145 ## 3-4. open 파라미터
146
147 `CANSocketOpenParams`는 다음 정보를 담는다.
148
149 - `port_name` : 열 포트 이름 예) `"can0"`
150 - `channel_config` : bitrate, mode, loopback, rx path, filter 설정
151 - `runtime` : timeout 동작용 tick / relax hook
152 - `hooks` : 선택적 이벤트 훅
153 - `flags` : 확장 플래그
154
155 ### 자주 쓰는 초기화 helper
156
157 - `CANSocketInitOpenParams()`
158 - `CANSocketInitOpenParamsClassic500k()`
159 - `CANSocketInitOpenParamsFd500k2M()`
160
161 보통은 아래 둘 중 하나로 시작하면 된다.
162
163 - Classical CAN 기본: `CANSocketInitOpenParamsClassic500k()`
164 - CAN FD + BRS 기본: `CANSocketInitOpenParamsFd500k2M()`
165
166 ---
167
168 ## 3-5. 가장 간단한 사용 예시
169
170 ```c
171 #include "can_socket.h"
172 #include "can_types.h"
173
174 static CANSocket g_socket;
175
176 void AppCanInit(void)
177 {
178     CANSocketOpenParams params;
179
180     CANSocketInit(&g_socket);
181     CANSocketInitOpenParamsClassic500k(&params);
182
183     params.port_name = "can0";
184
185     /* 필요 시 timeout hook 연결

```

```

186     params.runtime.get_tick_ms = AppGetTickMs;
187     params.runtime.relax = AppRelax;
188     params.runtime.user_context = 0;
189     */
190
191     (void)CANSocketOpen(&g_socket, &params);
192 }
193
194 void AppCanSendExample(void)
195 {
196     CANFrame frame;
197     uint8_t payload[4] = {0x11, 0x22, 0x33, 0x44};
198
199     CANFrameInitClassicStd(&frame, 0x123U, 4U);
200     (void)CANFrameSetData(&frame, payload, 4U);
201
202     (void)CANSocketSend(&g_socket, &frame);
203 }
204
205 void AppCanDeinit(void)
206 {
207     (void)CANSocketClose(&g_socket);
208 }
209 ...
210
211 ---
212
213 ## 3-6. 송신/수신 API
214
215 ### 송신 계열
216
217 - `CANSocketSend()`
218     기본 송신
219
220 - `CANSocketTrySend()`
221     즉시 시도. 지금 바로 불가능하면 busy 계열 상태를 받을 수 있다.
222
223 - `CANSocketSendNow()`
224     의미상 즉시 송신 helper
225
226 - `CANSocketSendTimeout()`
227     timeout을 둔 송신
228
229 - `CANSocketSendClassicStd()`
230     표준 ID Classical CAN 편의 함수
231
232 - `CANSocketSendClassicExt()`
233     확장 ID Classical CAN 편의 함수
234
235 - `CANSocketSendFdStd()`
236     표준 ID CAN FD 편의 함수
237
238 - `CANSocketSendFdExt()`
239     확장 ID CAN FD 편의 함수
240
241 ### 수신 계열
242
243 - `CANSocketReceive()`
244     기본 수신
245
246 - `CANSocketTryReceive()`

```

```

247     즉시 수신 시도
248
249     - `CANSocketReceiveNow()`
250     의미상 즉시 수신 helper
251
252     - `CANSocketReceiveTimeout()`
253     timeout을 둔 수신
254
255     - `CANSocketReceiveMatch()`
256     특정 조건을 만족하는 프레임을 받을 때까지 진행
257
258     - `CANSocketReceiveTimeoutMatch()`
259     timeout을 둔 match 수신
260
261     ---
262
263     ## 3-7. poll / readiness 관련 API
264
265     `CANSocket`은 readiness-style API도 제공한다.
266
267     - `CANSocketQueryEvents()`
268     - `CANSocketPoll()`
269     - `CANSocketWaitTxReady()`
270     - `CANSocketWaitRxReady()`
271
272     주로 사용하는 이벤트 마스크는 다음과 같다.
273
274     - `CAN_CORE_EVENT_TX_READY`
275     - `CAN_CORE_EVENT_RX_READY`
276     - `CAN_CORE_EVENT_ERROR`
277     - `CAN_CORE_EVENT_STATE`
278
279     예를 들어, 송신 가능한 시점만 보고 싶다면:
280
281     ```c
282     uint32_t ready_mask = 0U;
283
284     (void)CANSocketPoll(
285         &g_socket,
286         (uint32_t)CAN_CORE_EVENT_TX_READY,
287         0U,
288         &ready_mask
289     );
290
291     if ((ready_mask & (uint32_t)CAN_CORE_EVENT_TX_READY) != 0U)
292     {
293         /* 송신 가능 */
294     }
295     ...
296
297     ---
298
299     ## 3-8. 상태/오류 관련 API
300
301     - `CANSocketGetLastStatus()`
302     - `CANSocketIsOpen()`
303     - `CANSocketIsStarted()`
304     - `CANSocketGetErrorState()`
305     - `CANSocketRecover()`
306
307     즉, 단순 송수신뿐 아니라 현재 상태 점검과 복구 요청까지 `CANSocket` 레벨에서 어
308
309     ---
310
311     ## 3-9. timeout 사용 시 주의점

```

```

312
313 finite timeout을 쓰려면 runtime hook이 필요하다.
314
315 즉,
316
317 - `timeout_ms = 0`
318   immediate / non-blocking 의미
319
320 - `timeout_ms = CAN_TIMEOUT_INFINITE` 또는 finite timeout
321   실제 시간 기준 동작을 하려면 `get_tick_ms`가 있어야 한다.
322
323 권장 형태는 아래와 같다.
324
325 ```c
326 static uint32_t AppGetTickMs(void *user_context)
327 {
328     (void)user_context;
329     return /* 보드 tick ms */;
330 }
331
332 static void AppRelax(void *user_context)
333 {
334     (void)user_context;
335     /* 짧은 polling 간 완화 처리 */
336 }
337 ...
338
339 ---
340
341 ## 4. `CANContext` / `CANExecutor` / `CANService` 계층
342
343 이 계층은 `CANSocket`보다 한 단계 위의 orchestration 계층이다.
344
345 ### 이런 경우에 사용
346
347 - send / receive / poll 작업을 여러 개 쌓아 두고 순차 진행하고 싶다.
348 - 작업 단위를 명시적으로 관리하고 싶다.
349 - 동적할당 없이 operation pool을 돌리고 싶다.
350 - 나중에 Asio-like 사용감에 가까운 구조로 확장하고 싶다.
351
352 즉, 단순 송수신보다 **여러 작업의 진행 관리**가 중요할 때 쓴다.
353
354 ---
355
356 ## 4-1. 각 타입의 역할
357
358 ### `CANOperation`
359 작업 1개를 나타낸다.
360
361 - SEND
362 - RECEIVE
363 - POLL
364
365 `CANOperation`은 작업 정의 자체이고, transport 자원을 소유하지 않는다.
366
367 ### `CANExecutor`
368 여러 `CANOperation*`를 담고, 한 step씩 진행시키는 실행기다.
369
370 ### `CANContext`
371 `CANCore`와 `CANExecutor`를 묶는 얇은 facade다.
372
373 ### `CANService`
374 사용자 입장에서 가장 편한 상위 helper다.
375
376 - operation 확보
377 - 준비

```

```

378 - 제출
379 - 진행
380 - 반납
381
382 을 한 레벨에서 묶어 준다.
383
384 ---
385
386 ## 4-2. 권장 이해 순서
387
388 사용자 입장에서는 아래 순서로 보면 된다.
389
390 1. `CANOperation` : 작업 1개
391 2. `CANExecutor` : 작업 여러 개를 실행
392 3. `CANContext` : core + executor 묶음
393 4. `CANService` : acquire/release까지 포함한 가장 편한 상위 helper
394
395 실제 앱 코드에서는 `CANService` 중심으로 보는 것이 제일 편하다.
396
397 ---
398
399 ## 4-3. 가장 실용적인 사용 패턴
400
401 ### 준비 단계
402
403 1. `CANExecutor` 초기화
404 2. `CANContext` 초기화 후 `core + executor` 바인딩
405 3. `CANService` 초기화
406 4. `CANServiceBindContext()`
407
408 ### 실행 단계
409
410 1. `CANServiceSubmitSend()` 또는 `CANServiceSubmitReceive()` 또는 `CANSe
411 2. `CANServicePollOne()` / `CANServiceRunOne()` / `CANServiceDispatchO
412 3. 완료된 operation 결과 확인
413 4. `CANServiceReleaseOperation()`
414
415 ---
416
417 ## 4-4. `CANService` 사용 예시
418
419 ```c
420 #include "can_service.h"
421 #include "can_context.h"
422 #include "can_executor.h"
423 #include "can_operation.h"
424
425 static CANExecutor g_executor;
426 static CANOperation *g_executor_slots[4];
427
428 static CANContext g_context;
429
430 static CANService g_service;
431 static CANOperation g_service_ops[4];
432 static bool g_service_in_use[4];
433
434 void AppCanServiceInit(CANCore *core)
435 {
436     CANExecutorInit(&g_executor, g_executor_slots, 4U);
437     CANContextInit(&g_context);
438     CANContextBind(&g_context, core, &g_executor);
439
440     CANServiceInit(&g_service, g_service_ops, g_service_in_use, 4U);

```

```

441     CANServiceBindContext(&g_service, &g_context);
442 }
443
444 void AppCanServiceSendExample(void)
445 {
446     CANFrame frame;
447     CANOperation *op = 0;
448
449     CANFrameInitClassicStd(&frame, 0x321U, 2U);
450     frame.data[0] = 0xAAU;
451     frame.data[1] = 0x55U;
452
453     if (CANServiceSubmitSend(&g_service, &op, &frame, 0U) == CAN_STATU
454     {
455         while (CANOperationIsDone(op) == false)
456         {
457             (void)CANServiceRunOne(&g_service);
458         }
459
460         /* 결과 확인 */
461         (void)CANOperationGetResult(op);
462
463         CANServiceReleaseOperation(&g_service, op);
464     }
465 }
466 ...
467 ---
468
469
470 ## 4-5. `CANContext`를 직접 쓸 때
471
472 `CANService` 없이도 가능하다.
473
474 이 경우는 보통 아래처럼 쓴다.
475
476 1. `CANOperationInit()`
477 2. `CANContextPrepareSend()` / `CANContextPrepareReceive()` / `CANCont
478 3. `CANContextSubmit()`
479 4. `CANContextRunOne()` 또는 `CANContextPoll()`
480
481 즉, `CANContext`는 `CANService`보다 더 낮은 레벨의 수동 제어형 인터페이스라고
482
483 ---
484
485 ## 4-6. `CANExecutor`를 직접 쓸 때
486
487 `CANExecutor`는 사용자가 직접 operation 슬롯 배열을 제공해야 한다.
488
489 - 동적할당 없음
490 - operation 포인터만 관리
491 - 한 번에 최대 한 step씩 진행 가능
492
493 보통은 `CANContext`를 통해 간접적으로 만지게 되고, 앱에서 `CANExecutor`를 직접
494
495 ---
496
497 ## 5. `CANFrame` 사용법
498
499 사용자가 자주 직접 만지는 공용 데이터 구조는 `CANFrame`이다.
500
501 ### 초기화 helper
502
503 - `CANFrameInit()`

```



```

504 - `CANFrameInitClassicStd()`
505 - `CANFrameInitClassicExt()`
506 - `CANFrameInitFdStd()`
507 - `CANFrameInitFdExt()`
508
509 ### payload 설정
510
511 - `CANFrameSetData()`
512
513 ### 예시
514
515 ```c
516 CANFrame frame;
517 uint8_t payload[8] = {1,2,3,4,5,6,7,8};
518
519 CANFrameInitClassicStd(&frame, 0x100U, 8U);
520 (void)CANFrameSetData(&frame, payload, 8U);
521 ```
522
523 직접 `frame.data[]`를 채워도 되지만, 일관성을 위해 helper를 쓰는 편이 좋다.
524
525 ---
526
527 ## 6. 사용자 입장에서 알아야 할 상태 코드
528
529 대표적으로 아래 상태들만 우선 알아도 된다.
530
531 - `CAN_STATUS_OK` : 성공
532 - `CAN_STATUS_EINVAL` : 잘못된 인자 / 상태
533 - `CAN_STATUS_EBUSY` : 지금 바로 처리 불가 / 이미 사용 중
534 - `CAN_STATUS_ENODATA` : 수신 데이터 없음
535 - `CAN_STATUS_ETIMEOUT` : timeout 발생
536 - `CAN_STATUS_EUNSUPPORTED` : 현재 binding/driver에서 지원 안 함
537 - `CAN_STATUS_EIO` : 저수준 드라이버 오류
538
539 ---
540
541 ## 7. 내부 계층 요약
542
543 여기부터는 사용자가 직접 오래 붙잡고 있을 필요는 없지만, 구조 이해를 위해 짧게만 2
544
545 ## 7-1. `CANCore`
546
547 `CANCore`는 라이브러리의 안정 계층이다.
548
549 역할은 다음과 같다.
550
551 - open / close
552 - start / stop
553 - send / receive
554 - timeout-aware send / receive
555 - optional event query / error state / recover
556 - stats / last status 관리
557
558 사용자는 직접 써도 되지만, 보통은 `CANSocket`이나 `CANContext`를 통해 간접 사
559
560 ---
561
562 ## 7-2. `CANPlatform`
563
564 `CANPlatform`은 사용자 코드와 보드/드라이버 사이의 **binding 담당 계층**이다.
565
566 사용자가 알아야 하는 핵심만 정리하면:
567
568 1. `port_name` 예) `"can0"` 를 받는다.
569 2. 보드 계층에서 해당 이름의 실제 포트를 찾는다.

```

```

570 3. 현재 빌드 대상 보드/백엔드에 맞는 glue를 통해 `CANCoreBinding`을 만든다.
571 4. 그 binding으로 `CANCoreOpen()`을 호출한다.
572
573 즉, 사용자는 포트 이름만 넘기지만, 내부에서는 다음이 자동으로 연결된다.
574
575 - 어떤 드라이버 ops를 쓸지
576 - 어떤 driver channel storage를 쓸지
577 - 어떤 board port metadata를 쓸지
578 - 현재 포트가 FD/BRS/filter/termination을 지원하는지
579
580 ### binding이 중요한 이유
581
582 `CANCore`는 특정 MCU 드라이버를 직접 알지 않는다.
583 대신 `CANCoreBinding`을 통해 아래 정보만 받는다.
584
585 - 필수 driver ops
586 - 선택 optional ops
587 - driver channel 포인터
588 - driver port 포인터
589 - capabilities
590
591 즉, **Core는 binding을 통해 platform-specific 구현과 연결**된다.
592
593 그래서 플랫폼을 바꾸더라도, 사용자 관점 API는 가능한 한 유지되고, binding만 바뀌
594
595 ---
596
597 ## 7-3. MCU Driver 계층
598
599 예를 들면:
600
601 - TC375 계열: `can_tc3xx`
602 - TC275 계열: `can_tc275`
603
604 이 계층은 실제 CAN 컨트롤러 레지스터/iLLD 동작과 가까운 부분이다.
605
606 사용자는 보통 직접 만지지 않는다.
607
608 ---
609
610 ## 7-4. BSP / Board 계층
611
612 이 계층은 보드별 포트 정의와 핀 정보를 제공한다.
613
614 예를 들면:
615
616 - 포트 이름
617 - 어떤 CAN 인스턴스/노드인지
618 - RX/TX/STB 핀
619 - FD 지원 여부
620 - termination 제어 가능 여부
621
622 즉, `"can0"` 같은 이름을 실제 하드웨어 포트로 해석해 주는 기반 계층이다.
623
624 ---
625
626 ## 8. 어떤 계층을 선택하면 좋은가
627
628 ### 8-1. 일반 앱 코드
629
630 **`CANSocket` 추천**
631
632 - 가장 짧다.
633 - 가장 직관적이다.
634 - open / close까지 포함해 바로 쓸 수 있다.
635
636 ### 8-2. 작업 여러 개를 정적으로 관리해야 할 때

```

```

637
638 **`CANService` 추천**
639
640 - operation 확보/반납까지 포함되어 편하다.
641 - `CANContext` / `CANExecutor`를 직접 다루는 것보다 앱 코드가 덜 지저분하다.
642
643 ### 8-3. 더 낮은 레벨 제어가 필요할 때
644
645 - `CANContext`
646 - `CANExecutor`
647 - `CANOperation`
648
649 순으로 내려가면 된다.
650
651 ### 8-4. 라이브러리 내부 확장 또는 플랫폼 포팅
652
653 - `CANCore`
654 - `CANPlatform`
655 - binding glue
656 - MCU driver
657
658 쪽을 보면 된다.
659
660 ---
661
662 ## 9. 추천 사용 흐름 요약
663
664 ### 가장 간단한 경로
665
666 ```text
667 CANSocketInit
668
669 -> CANSocketInitOpenParamsClassic500k / Fd500k2M
670
671 -> params.port_name 설정
672
673 -> CANSocketOpen
674
675 -> CANSocketSend / Receive / Poll
676 -> CANSocketClose
677 ```
678
679 ### 작업 orchestration 경로
680
681 ```text
682 CANExecutorInit
683
684 -> CANContextInit + CANContextBind
685
686 -> CANServiceInit + CANServiceBindContext
687
688 -> CANServiceSubmitSend / Receive / Poll
689
690 -> CANServiceRunOne 또는 CANServicePoll
691
692 -> CANOperation 결과 확인
693 -> CANServiceReleaseOperation
694 ```
695
696 ---
697
698 ## 10. 결론
699
700 이 라이브러리를 사용자 관점에서 보면 핵심은 아래 두 줄로 정리된다.
701
702 1. **보통은 `CANSocket`을 쓰면 된다.**
703     단순 송수신, poll, timeout, 상태 확인까지 가장 쉽게 접근 가능하다.
704
705 2. **여러 작업을 정적으로 관리하려면 `CANService` 계열을 쓰면 된다.**

```

```
697     그 아래의 `CANContext` / `CANExecutor` / `CANOperation`은 orchestrati
698
699     그리고 내부적으로는 `CANPlatform`이 포트 이름과 보드 정보를 기반으로 **binding
```

CAN 라이브러리 사용자 계층 인터페이스 명세

이 문서는 현재 CAN 라이브러리에서 사용자가 실제로 직접 사용할 계층을 중심으로 정의한 문서이다. 구현 내용은 *CANCore* 이하 계층은 현재 구조를 이해할 수 있을 정도라면 간략히 설명한다.

문서의 기준은 다음과 같다.

- 가장 간단한 사용 경로: *CANSocket*
- 여러 작업을 정적으로 관리하는 상위 경로: *CANContext* / *CANExecutor* / *CANService*
- 내부 구현 계층: *CANCore* / *CANPlatform* / *MCU Driver* / *ESP*

1. 권장 사용 계층

실사용 관점에서는 이해처럼 생각하면 된다.

1-1. 가장 쉬운 사용법

- CANSocket*
- 간단 포트 열기 / 송신 / 수신 / poll / timeout 기반 동작을 가장 간단하게 쓸 수 있는 계층이다.
- 다루분의 매물지키이션은 우선 이 계층부터 사용하면 된다.

1-2. 여러 작업을 정적으로 관리하고 실행 때

- CANContext* + *CANExecutor* + *CANService*
- send/receive/poll 작업을 여러 개 관리해 두고, 정적 실행 안에서 순차적으로 진행시키고 실행 때 사용한다.
- 비동기 프래임워크처럼 보이지만, 실제 모델은 동적할당 없는 *polling* 기반 *operation orchestration*이다.

1-3. 내부 실행 계층

- CANCore*
- 실제 송수신, 상태 관리, timeout-aware polling의 기준이 되는 실행 계층이다.
- 사용자로 직접 쓸 수는 있지만, 일반적인 앱 레벨에서는 *CANSocket* 또는 *CANContext* 계층을 먼저 쓰는 편이 낫다.

2. 전체 계층 구조

```
graph TD
    subgraph USER [User-facing layer]
        SOCKET[CANSocket]
        SERVICE[CANService]
        CONTEXT[CANContext]
        EXECUTOR[CANExecutor]
        OP[CANOperation]
    end

    subgraph CORE [Core layer]
        CORENODE[CANCore]
    end

    subgraph PLATFORM [Platform layer]
        PLATFORMNODE[CANPlatform]
        GLUE[Board/backend specific binding glue]
    end

    subgraph MCU [MCU Driver layer]
        DRIVER[can_is1304 / can_is1379]
    end

    subgraph ESP [ESP / Board layer]
        BOARD[board_api / board port map]
    end

    APP --> SOCKET
    APP --> SERVICE
    SERVICE --> CONTEXT
    CONTEXT --> EXECUTOR
    CONTEXT --> OP
    SOCKET --> CORENODE
    CONTEXT --> CORENODE
    CORENODE --> PLATFORMNODE
```

```
PLATFORMIO ==> MCU
MCU ==> DRIVER
PLATFORMIO ==> BOARD
```

계층별 역할 요약

- **Application**
사용자 코드.
- **CANSocket**
가장 쉽고 쉬운 공개 인터페이스, 단순 송수신용.
- **CANService / CANContext / CANExecutor / CANOperation**
여러 작업을 정책 차원으로 관리하는 상위 orchestration 계층.
- **CANCore**
라이브러리의 실행 로직, 실제 send / receive / poll / timeout 처리 계층.
- **CANPlatform**
포트 이름("can0")을 실제 포트 번호로 찾고, 해당 포트/맥언트어 맞는 **binding** 을 구성해서 **CANCore**에 연결한다.
- **MCU Driver layer**
TC275면 `can_bcdm`, TC275면 `can_tc275` 같은 실제 드라이버 레이어.
- **BSP / Board layer**
보드별 포트 정의, 핀맵, PinTermination 정보 제공.

3. CANSocket 계층

3-1. 언제 쓰면 되는가

CANSocket은 다음 같은 경우에 적합하다.

- 코드 하나 열어서 바로 송신/수신하고 싶다.
- `open -> send/receive -> close` 흐름이 필요하다.
- 복잡한 operation pool 없이 가장 높은 API를 쓰고 싶다.
- polling-first 스타일로 동작해도 된다.

즉, 일반적인 사용자를 1순위 API라고 보면 된다.

3-2. 핵심 특징

- **CANCore** 위에 얹은 socket-like facade
- 동적할당 없음
- 두 가지 사용 방식 지원
 - **blocking open** `CANSocketOpen()` 으로 내부 core까지 직접 관리
 - **non-blocking bind** 외부에서 만든 **CANCore**를 `CANSocketBindCore()` 로 연결

가장 단순한 경우에는 `blocking open`만 하면 된다.

3-3. 기본 수행주기

가장 일반적인 흐름

1. `CANSocketInit()`
2. `CANSocketInitOpenParamClass()` 또는 `CANSocketInitOpenParamDefault()`
3. `param.port_name = "can0"` 설정
4. 필요 시 runtime timeout hook 설정
5. `CANSocketOpen()`
6. `CANSocketSend()` / `CANSocketReceive()` / `CANSocketPoll()` 사용
7. `CANSocketClose()`

3-4. open 파라미터

`CANSocketOpenParam`는 다음 정보를 넣는다.

- `port_name`: 열 포트 이름 예) "can0"
- `channel_config`: bitrate, mode, loopback, rx path, filter 설정
- `pollSec`: timeout 동작을 tick / rtos hook
- `hooks`: 선택적 이벤트 콜

- `Flags` : 확장 플래그

자주 쓰는 초기화 helper

- `CANSocketInitOpenParams()`
- `CANSocketInitOpenParamsClassic1000()`
- `CANSocketInitOpenParamsFD500K2M()`

보통은 아래 중 하나로 시작하면 된다.

- Classical CAN 기본: `CANSocketInitOpenParamsClassic1000()`
- CAN FD + BRS 기본: `CANSocketInitOpenParamsFD500K2M()`

3-5. 가장 간단한 사용 예시

```
#include "can_socket.h"
#include "can_types.h"

static CANSocket g_socket;

void AppCanInit(void)
{
    CANSocketOpenParams param;

    CANSocketInit(&g_socket);
    CANSocketInitOpenParamsClassic1000(&param);

    param.port_name = "can0";

    /* 필요 시 timeout hook 연결
    param.runtime_get_tick_ms = AppGetTicks;
    param.runtime_mutex = AppMutex;
    param.runtime_user_context = 0;
    */

    (void)CANSocketOpen(&g_socket, &param);
}

void AppCanSendExample(void)
{
    CANFrame frame;
    uint8_t payload[4] = {0x10, 0x20, 0x30, 0x40};

    CANFrameInitClassicStd(&frame, 0x123, 4);
    (void)CANFrameSetData(&frame, payload, 4);

    (void)CANSocketSend(&g_socket, &frame);
}

void AppCanClose(void)
{
    (void)CANSocketClose(&g_socket);
}
```

3-6. 송신/수신 API

송신 계열

- `CANSocketSend()`
기본 송신
- `CANSocketTrySend()`
특시 시도, 지금 바로 불가능하면 busy 계통 상태를 받을 수 있다.
- `CANSocketSendNow()`
최악상 특시 송신 helper
- `CANSocketSendTimeout()`
timeout을 둔 송신
- `CANSocketSendClassicStd()`
표준 ID-Classical CAN 범위 함수

- `CANSocketSendClassical()`
특정 ID-Classical CAN 메시지 전송
- `CANSocketSendFdx()`
표준 ID-CAN FD 메시지 전송
- `CANSocketSendPdu()`
특정 ID-CAN FD 메시지 전송

수신 계열

- `CANSocketReceive()`
기본 수신
- `CANSocketTryReceive()`
즉시 수신 시도
- `CANSocketReceiveNow()`
원치않 즉시 수신 helper
- `CANSocketReceiveTimeout()`
timeout을 둔 수신
- `CANSocketReceiveMatch()`
특정 조건을 만족하는 프레임들을 받을 때까지 대기
- `CANSocketReceiveTimeoutMatch()`
timeout을 둔 match 수신

3-7. poll / readiness 관련 API

`CANSocket`은 readiness-style API도 제공한다.

- `CANSocketQueryEvents()`
- `CANSocketPoll()`
- `CANSocketWaitTtReady()`
- `CANSocketWaitFdReady()`

주로 사용하는 이벤트 마스크는 다음과 같다.

- `CAN_CORE_EVENT_TX_READY`
- `CAN_CORE_EVENT_RX_READY`
- `CAN_CORE_EVENT_ERROR`
- `CAN_CORE_EVENT_STATE`

여를 통해, 송신 가능한 시점만 보고 싶다면

```
uint32_t ready_mask = 0U;

(void)CANSocketPoll(
    &g_socket,
    (uint32_t)CAN_CORE_EVENT_TX_READY,
    0U,
    &ready_mask
);

if ((ready_mask & (uint32_t)CAN_CORE_EVENT_TX_READY) != 0U)
{
    /* 송신 가능 */
}
```

3-8. 상태/오류 관련 API

- `CANSocketGetLastStatus()`
- `CANSocketIsOpen()`
- `CANSocketIsStarted()`
- `CANSocketIsNonCata()`
- `CANSocketRecover()`

특, 많은 송수신뿐 아니라 현재 상태 점검과 복구 요청까지 `CANSocket` 레벨에서 어느 정도 처리할 수 있다.

3-9. timeout 사용 시 주의점

finite timeout을 쓰려면 runtime flag가 필요하다.

즉,

- `timeout_ms = 0`
immediate / non-blocking 호출
- `timeout_ms = CAN_TIMEOUT_INFINITE` 또는 finite timeout
일정 시간 가운 동작을 하려면 `get_tick_ms`가 있어야 한다.

문장 형태는 아래와 같다.

```
static void_t AppletTickMs(void *user_context)
{
    (void)user_context;
    return /* 500 tick ms */;
}

static void AppletIax(void *user_context)
{
    (void)user_context;
    /* 모든 polling 간 변화 처리 */
}
```

4. CANContext / CANExecutor / CANService 계층

이 계층은 `CANSocket`보다 한 단계 위의 orchestration 계층이다.

이런 경우에 사용

- send / receive / poll 작업을 여러 개 실행 하고 순차 진행하고 싶다.
- 작업 항목을 명시적으로 관리하고 싶다.
- 동적일당 없이 operation pool을 만들고 싶다.
- 나중에 Auto-like 사용법에 가까운 구조로 확장하고 싶다.

즉, 단순 송수신보다 여러 작업의 진행 관리가 필요할 때 쓴다.

4-1. 각 타업의 역할

CANOperation

작업 1개를 나타낸다.

- SEND
- RECEIVE
- POLL

CANOperation은 작업 정의 자체이고, transport 지정을 사용하지 않는다.

CANExecutor

여러 **CANOperation***를 넣고, 한 step씩 진행시키는 실행기다.

CANContext

CANCore은 **CANExecutor**를 묶는 넓은 scope다.

CANService

사용자 입장에서 가장 완전 상위 layer다.

- operation 확보
- 준비
- 제출
- 진행
- 반납

을 한 레벨에서 묶어 준다.

사용자 입장에서선 아래 순서로 보면 된다.

1. `CANOperation` : 작업 5개
2. `CANExecutor` : 작업 여러 개를 실행
3. `CANContext` : core + executor 묶음
4. `CANService` : acquire/release까지 포함한 가장 완전 상위 helper

실제 앱 코드에서는 `CANService` 용성으로 보는 것이 제일 편하다.

4-3. 가장 실용적인 사용 패턴

준비 단계

1. `CANExecutor` 초기화
2. `CANContext` 초기화 후 (core + executor) 매핑
3. `CANService` 초기화
4. `CANServiceBindContext()`

실행 단계

1. `CANServiceSubmitSend()` 또는 `CANServiceSubmitReceive()` 또는 `CANServiceSubmitPoll()`
2. `CANServicePollDone()` / `CANServiceRunDone()` / `CANServiceInputDone()` / `CANServicePoll()`
3. 완료된 operation 결과 확인
4. `CANServiceReleaseOperation()`

4-4. CANService 사용 예시

```
#include "can_service.h"
#include "can_context.h"
#include "can_executor.h"
#include "can_operation.h"

static CANExecutor g_executor;
static CANOperation *g_executor_ops[4];

static CANContext g_context;

static CANService g_service;
static CANOperation g_service_ops[4];
static bool g_service_in_use[4];

void AppCanServiceInit(CANCore *core)
{
    CANExecutorInit(&g_executor, g_executor_ops, 4);
    CANContextInit(&g_context);
    CANContextBind(&g_context, core, &g_executor);

    CANServiceInit(&g_service, g_service_ops, g_service_in_use, 4);
    CANServiceBindContext(&g_service, &g_context);
}

void AppCanServiceSendSample(void)
{
    CANFrame frame;
    CANOperation *op = 0;

    CANFrameGetClassID(&frame, 0x333, 20);
    frame.data[0] = 0xA55;
    frame.data[1] = 0x555;

    if (CANServiceSubmitSend(&g_service, op, &frame, 0) == CAN_STATUS_OK)
    {
        while (CANOperationIsDone(op) == false)
        {
            (void)CANServiceRunDone(&g_service);
        }

        /* 결과 확인 */
        (void)CANOperationGetResult(op);

        CANServiceReleaseOperation(&g_service, op);
    }
}
```

```
)
```

4-5. CANContext를 직접 쓸 때

CANService 없이도 가능하다.

이 경우는 보통 아래처럼 쓴다.

```
1. CANOperationInit();
2. CANContextPrepareLoad() / CANContextPrepareReceive() / CANContextPreparePoll();
3. CANContextSubmit();
4. CANContextRunOne(); 또는 CANContextPoll();
```

즉, CANContext는 CANService보다 더 낮은 레벨의 주를 제어할 인터페이스라고 보면 된다.

4-6. CANExecutor를 직접 쓸 때

CANExecutor는 사용자가 직접 operation 실행 배열을 제공하여 한다.

- 동적할당 없음
- operation 고정치만 관리
- 한 번에 최대 한 step씩 진행 가능

보통은 CANContext를 통해 간접적으로 관리가 되고, 없어서 CANExecutor를 직접 강력게 제어하는 경우는 상대적으로 적다.

5. CANFrame 사용법

사용자가 자주 직접 만지는 용을 제어해 주려는 CANFrame이다.

초기화 helper

- CANFrameInit();
- CANFrameInitClassic();
- CANFrameInitClassicExt();
- CANFrameInitFifo();
- CANFrameInitFifoExt();

payload 설정

- CANFrameSetData();

예시

```
CANFrame frame;
uint8_t payload[8] = {1,2,3,4,5,6,7,8};

CANFrameInitClassic(&frame, 0x100, 0x);
(void)CANFrameSetData(&frame, payload, 0x);
```

직접 frame.data[]를 써줘도 되지만, 일관성을 위해 helper를 쓰는 편이 좋다.

6. 사용자 입장에서 알아야 할 상태 코드

대략적으로 아래 상태들만 우선 알아도 된다.

- CAN_STATUS_OK : 성공
- CAN_STATUS_ERROR : 잘못된 입자 / 상태
- CAN_STATUS_BUSY : 지금 바로 처리 불가 / 이미 사용 중
- CAN_STATUS_TIMEOUT : 우선 데이터 없음
- CAN_STATUS_TIMEOUT : timeout 발생
- CAN_STATUS_UNSUPPORTED : 현재 binding/driver에서 지원 안 함
- CAN_STATUS_ERR : 저수준 트라이버 오류

7. 내부 계층 요약

여기부터는 사용자가 직접 오해 불필요 있을 필요는 없지만, 구조 이해를 위해 짧게만 정리한다.

7-1. CANCore

CANCore는 라이브러리의 핵심 계층이다.

역할은 다음과 같다.

- open / close
- start / stop
- send / receive
- timeout-secure send / receive
- optional event query / error state / recover
- state / last status 관리

사용자는 직접 써도 되지만, 보통은 **CANSocket**이나 **CANContext**를 통해 간접 사용하게 된다.

7-2. CANPlatform

CANPlatform은 사용자 포트와 보드/드라이버 사이의 **binding** 담당 계층이다.

사용자가 알아야 하는 핵심한 정리하면:

1. port_name 즉 "can0"를 얻는다.
2. 보드 계층에서 해당 이름의 실제 포트를 찾는다.
3. 현재 보드 대상 보드/백엔드로 맞는 glue를 통해 **CANCoreBinding**을 만든다.
4. 그 binding으로 **CANCoreOpen()**을 호출한다.

즉, 사용자는 포트 이름을 알기지만, 내부에서는 다음이 자동으로 연결된다.

- 어떤 드라이버 ops를 쓸지
- 어떤 driver channel storage를 쓸지
- 어떤 board/port metadata를 쓸지
- 현재 포트가 RX/TX/Filter/Termination을 지원하는지

binding이 필요한 이유

CANCore는 특정 MCU 드라이버를 직접 알지 않는다.

대신 **CANCoreBinding**을 통해 아래 정보를 받는다.

- 필수 driver ops
- 선택 optional ops
- driver channel 포인터
- driver port 포인터
- capabilities

즉, **Core**는 **binding**을 통해 **platform-specific** 구현과 연결된다.

그래서 플랫폼을 바꾸더라도, 사용자 관점 API는 가능한 한 유지되고, binding만 바뀌는 구조다.

7-3. MCU Driver 계층

역할 설명:

- TCI2PS 계층: **can_tci2ps**
- TCI2PS 계층: **can_tci2ps**

이 계층은 실제 CAN 컨트롤러 레지스터/IO와 물리적 가락을 구현한다.

사용자는 보통 직접 쓰지 않는다.

7-4. BSP / Board 계층

이 계층은 보드별 포트 정의와 관련 정보를 제공한다.

역할 설명:

- 포트 이름
- 어떤 CAN 인스턴스/노드인지
- RX/TX/GP비 핀
- RS-지침 여부
- termination 제어 가능 여부

즉, "can0" 같은 이름을 실제 하드웨어 포트로 해석해 주는 기반 계층이다.

8. 어떤 계층을 선택하면 좋은가

8-1. 일반 입 로트

CANSocket 추천

- 가장 쉽다.
- 가장 직관적이다.
- open / close까지 포괄해 바로 쓸 수 있다.

8-2. 작업 여러 가를 정적으로 관리해야 할 때

CANService 추천

- operation 확보/병합까지 포함되게 명확하다.
- CANContext / CANExecutor를 직접 다루는 것보다 입 로트가 덜 지저분하다.

8-3. 더 낮은 레벨 제어가 필요할 때

- CANContext
- CANExecutor
- CANOperation

순으로 내려가면 된다.

8-4. 라이브러리 내부 확장 또는 플랫폼 포팅

- CANCore
- CANPlatform
- binding glue
- MCU driver

책을 보면 된다.

9. 추천 사용 흐름 요약

가장 간단한 경로

```
CANSocketInit
-> CANSocketInitOpenParamsClassic4096 / Fd4096m
-> param.port_name 설정
-> CANSocketOpen
-> CANSocketSend / Receive / Poll
-> CANSocketClose
```

작업 orchestration 경로

```
CANExecutorInit
-> CANContextInit = CANContextBind
-> CANServiceInit = CANServiceBindContext
-> CANServiceSubInitSend / Receive / Poll
-> CANServiceRunOne 또는 CANServicePoll
-> CANOperation 결과 확인
-> CANServiceDeInitOperation
```

10. 결론

이 라이브러리를 사용자 관점에서 보면 핵심은 아래 두 줄로 정리된다.

- 보통은 CANSocket 을 쓰면 된다.
단순 송수신, poll timeout, 상태 확인까지 가장 쉽게 접근 가능하다.
- 여러 작업을 정적으로 관리하려면 CANService 계층을 쓰면 된다.
그 이외의 CANContext / CANExecutor / CANOperation은 orchestration을 위한 내부 구성 블록에 거깝다.

그리고 내부적으로는 CANPlatform이 로트 이름과 보드 정보를 기반으로 binding 을 만들어 CANCore와 실제 드라이버를 연결한다고 이해하면 된다.

